Project 1: Robot Code Documentation

Bryan Tran, Donovan Semenuk

1 Controller Architecture

The robot controller is a ROS 2 node following a pseudo-subsumption reactive architecture. A single `tick()` method runs at 20 Hz and evaluates each behavior in priority order, returning immediately when one fires. This `return`-based suppression gives the layered, sensor-driven behavior of subsumption without the complexity of wiring explicit inhibition signals between separate layers. Each higher-priority behavior implicitly overrides all behaviors below it simply by exiting the loop early.

This architecture was chosen because it was discussed and shown in class. A schema-based approach was also considered but we ultimately chose to go with the one above because the other was more familiar.

The behaviors, from highest to lowest priority, are: (1) Halt, (2) Keyboard, (3) Escape, (4) Avoid, (5) Random turn every 1 ft, (6) Drive forward.

2 ROS Communication

The node publishes `Twist` messages to `/cmd_vel`, which the robot's motor driver consumes as linear and angular velocity commands. It subscribes to three topics: `/cmd_vel_key` for keyboard teleop input, `/odom` for position and orientation as `Odometry` messages, and `/scan` for lidar range data as `LaserScan` messages.

Each subscription callback updates shared instance variables, flags, minimums, and pose estimates, that `tick()` reads on every control cycle. This separates the timing of sensing from the timing of acting: sensor data is processed whenever it arrives, and the control loop acts on the most recent snapshot each tick.

3 Data Structures and Algorithms

Data Structures

In terms of data structures, the controller primarily uses simple state variables stored within the `Project1Controller` class to maintain the robot's current state. We found that other data structures, like arrays, hash maps, etc. were not needed for the scope of this project. `halt_detected` and `collision_detected` are Boolean flags set in `scan_callback` when any beam falls below `HALT_DIST` (0.20 m) or `COLLISION_DIST` (0.45 m) respectively. `front_left_min` and `front_right_min` store the minimum range within the $\pm 30°$ front arc, split by the sign of the beam angle to support left/right asymmetry detection.

`dist_since_turn` accumulates the Euclidean distance $\sqrt{\Delta x^2 + \Delta y^2}$ between successive odometry poses, giving an estimate of travel since the last turn.

Robot orientation is stored as a quaternion. The code for this was based on "Project_1_Guide.pdf" that was provided.

$$\text{yaw} = \text{atan2}\big(2(wz + xy),\ 1 - 2(y^2 + z^2)\big)$$

Implementation:

```
def quat_to_yaw(q):
    siny = 2*(q.w*q.z + q.x*q.y)
```

```
    cosy = 1 - 2*(q.y*q.y + q.z*q.z)
    return math.atan2(siny, cosy)
```

Helper Algorithms

Angle Arithmetic

`yaw_remaining(target, current)` computes the signed angular error between the robot's current heading and a goal heading, wrapped into $(-\pi, \pi]$ via modular arithmetic:

```
return (target - current + math.pi) % (2 * math.pi) - math.pi
```

This ensures the robot always rotates the shorter way and never over-shoots past $\pm 180°$. Turn commands apply `TURN_SPEED` with the sign of the remaining error until the magnitude drops below a $2°$ tolerance.

Lidar Handling

In `scan_callback`, beam angles are computed from the scan header rather than assumed fixed: `angle = angle_min + i * angle_increment`. Beams are then wrapped to $(-\pi, \pi]$ and classified as left (angle ≥ 0) or right (angle < 0) within the front arc. For the asymmetry test, infinite readings (no obstacle) are replaced by a sentinel value of $2 \times$ `TURN_DIST` so an open side never falsely appears closer than an obstructed side.

Behavior Algorithms

Behavior algorithms are discussed below in Section 4.

4 Behavior Implementation

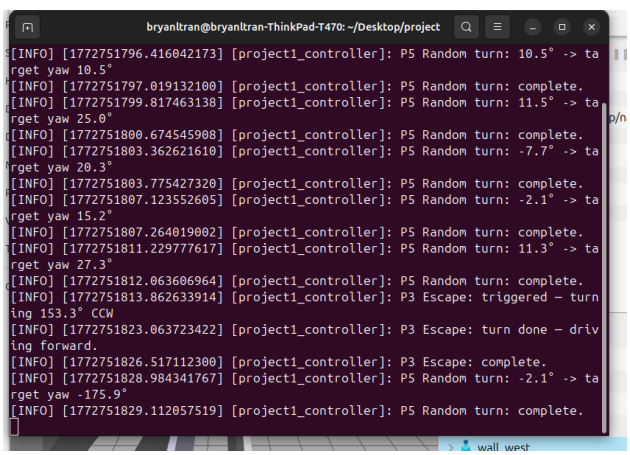

Figure 1: Behavior Logging in Terminal

Behavior 1: Halt

If `halt_detected` is true, a zero-velocity `Twist` is published and `tick()` returns immediately. The `return` statement is the architectural suppression mechanism, no lower-priority behavior can execute while this condition holds.

Behavior 2: Keyboard

When `key_active` is true, the most recent `/cmd_vel_key` command is forwarded directly to `/cmd_vel`. A 0.3 s timeout (`KEY_TIMEOUT`) clears `key_active` after the user releases the keys, returning the robot to autonomous control. All lower-priority behavior flags are reset on keyboard entry for a clean handoff.

Behavior 3: Escape

Escape is a *fixed-action pattern*: once `collision_detected` triggers it, the two-phase state machine runs to completion regardless of subsequent sensor changes. This is a fixed-action pattern, where the stimulus releases the behavior but does not need to persist to sustain it. There is also a cooldown of 5 seconds on this behavior because there was an issue during development where the bot would be stuck in this loop of escape and not get out of it. As seen in Figure 1, the console logs whenever escape triggers and the phases of escape.

Behavior 4: Avoid

Avoid handles *asymmetric* obstacles detected in the front arc. When either side is within `TURN_DIST` (1 ft) and the left/right minima differ by more than `ASYM_THRESHOLD`, the controller initiates a short avoidance maneuver by selecting a random angle $\theta \in [30°, 60°]$ and turning away from the closer side. This behavior is implemented as a bounded turn-to-heading action: once started, the robot turns in place until it reaches the computed `avoid_target_yaw` (within 2°), then resumes lower-priority motion.

Behavior 5: Random Turn Every 1 ft

After each foot of forward travel, the controller draws a uniform random angle from $[-15°, 15°]$, sets a new target yaw, and rotates to it in place before resuming forward motion. `dist_since_turn` is reset whenever this behavior fires or any higher-priority behavior preempts it, so the interval is always measured from the last real turn. As seen in Figure 1, the terminal logs every time the bot randomly turns and the degree.

Behavior 6: Drive Forward

The lowest-priority behavior: if nothing above is active, the robot moves forward at `FORWARD_SPEED` (0.2 m/s) with zero angular velocity, continuing to explore the environment indefinitely.